



Referência Técnica

Para programadores

Documentação técnica e operacional do sistema RedeApoio

Gerado em **04 de agosto de 2026**

Origem: **docs/04-referencia-tecnica.md**

Este PDF é gerado automaticamente a partir da documentação do repositório. A versão sempre atual está em **docs/04-referencia-tecnica.md**, no GitHub — em caso de divergência, vale o que está lá.

4 — Referência técnica

Estado completo do sistema para quem vai mexer no código — pessoa ou agente. Leia isto antes de alterar qualquer coisa.

Stack

Camada	Escolha	Observação
Runtime	Node.js 20 (alpine)	<code>backend/Dockerfile</code>
API	Express 4	Express 4 não encaminha rejeição de Promise sozinho — ver <code>asyncHandler</code>
Banco	PostgreSQL 16	UUID nativo, <code>pgcrypto</code> para <code>gen_random_uuid()</code>
Auth	JWT em cookie httpOnly	12 h, <code>jsonwebtoken</code> + <code>bcryptjs</code>
Frontend	HTML + CSS + JS puro, arquivo único	<code>frontend/index.html</code> , ~2.440 linhas, sem build
E-mail	Nodemailer / SMTP	só recuperação de senha
Orquestração	Docker Swarm	não é Compose comum
Proxy / TLS	Traefik + Let's Encrypt	<code>infra/traefik-stack.yml</code>
Painel	Portainer CE	<code>infra/portainer-stack.yml</code>

Sem framework de frontend, sem bundler, sem TypeScript, sem suíte de testes. Isso é deliberado: o sistema é operado por quem não é programador, e um único arquivo estático elimina toda a superfície de build.

Mapa dos arquivos

docker-compose.yml	stack de produção (Portainer)
build.sh	builda a imagem no servidor e força o redeploy
migrate.sh	importação única do Supabase – histórico, não usar mais
.env.example	referência das variáveis
infra/traefik-stack.yml	proxy reverso + SSL
infra/portainer-stack.yml	painel de administração
scripts/backup-db.sh	backup diário (cron)
scripts/restore-db.sh	restauração / migração
backend/	
Dockerfile	copia backend/src, migrations, scripts e frontend/
migrations/001_init.sql	schema COMPLETO – reaplicado a cada boot
src/	
server.js	bootstrap: migrações → rotas → estático → listen
config.js	porta, JWT, limites padrão, versão do termo
db.js	pool do pg + parser de DATE
middleware/	
auth.js	authRequired, requireRole
workspace.js	impersonação do admin via ?as=<candidatoId>
routes/	
auth.js	login, /me, logout, esqueci/redefinir senha
usuarios.js	CRUD de liderança/apoiador (candidato/admin)
apoiadores.js	pirâmide: árvore, hierarquia, permissões ← núcleo
admin.js	candidatos, planos, contrato (só admin)
public.js	autocadastro por link (SEM autenticação)
conta.js	autoatendimento: senha, login, aceite do termo
config.js	limites da pirâmide por candidato
services/mail.js	SMTP + template de recuperação
utils/	
asyncHandler.js	obrigatório em toda rota async
duplicidade.js	telefone/título/e-mail repetidos na mesma rede
limites.js	limite do candidato, com fallback global
nivelUsuario.js	nível real de quem está logado
password.js	bcrypt + senha temporária via CSPRNG
termoStatus.js	precisa aceitar termo? trocar senha?
tituloEleitoral.js	dígito verificador do título (mod 11)
frontend/index.html	TUDO do frontend
docs/	esta documentação

Modelo de dados

Três tabelas. Todo o schema está em `backend/migrations/001_init.sql`.

`usuarios` — quem tem login

`id` (UUID), `nome`, `login` (único), `senha_hash`, `perfil` (`admin` | `candidato` | `lideranca` | `apoiador`), `criado_por`, `email`, dados de contato e eleitorais, `ativo`, `senha_temporaria`, `termo-versao-aceita`, `reset_password_token` / `_expires`, `plano`, `periodo_contrato`, `data_desativacao`, `limite_nivel1..4`.

`apoiadores` — todo mundo na pirâmide

`id` (UUID), dados pessoais, `nivel` (1 a 4), `parent_id` (quem é o responsável), `cadastrado_por`, `lgpd-aceite` / `_em` / `_versao`.

`termos-aceite` — trilha de auditoria da LGPD

Uma linha por aceite, **nunca sobrescrita**: `usuario_id` ou `apoiador_id` (exatamente um dos dois, garantido por CHECK), `versao-termo`, `aceite-em`, `ip`, `user-agent`.

As invariantes que não podem ser quebradas

1. `apoiadores.id == usuarios.id` para quem tem login

Esta é a regra mais importante do sistema.

Toda liderança e todo apoiador com login existe em **duas** tabelas: em `usuarios` (o acesso) e em `apoiadores` (a ficha na pirâmide). As duas linhas **compartilham o mesmo UUID**.

Por quê: quem essa pessoa indica grava `apoiadores.parent_id = <id do usuário>`. Se a ficha-espelho tivesse um id próprio (um `gen_random_uuid()` qualquer), nenhuma consulta conseguiria ligar a pessoa aos indicados dela — todo mundo apareceria com "0 indicados". **Foi exatamente esse o bug que existiu no sistema original**, e as linhas 71-79 do `001_init.sql` são o reparo dos dados antigos.

Consequência prática: **qualquer código novo que crie um usuário com login precisa inserir a ficha em `apoiadores` passando o id explicitamente**. Nunca deixe o default gerar.

Os três lugares que fazem isso hoje — use-os como modelo:

- `routes/usuarios.js` (candidato cria liderança/apoiador)
- `routes/public.js` (autocadastro de nível ≤ 3)
- `001_init.sql` (reparo de dados legados)

2. 001_init.sql roda em TODO boot e precisa ser idempotente

Não existe sistema de versionamento de migração. O `server.js` lê o arquivo inteiro e executa a cada inicialização do container.

Portanto, **toda instrução nova precisa poder rodar mil vezes sem efeito colateral**:

- `CREATE TABLE IF NOT EXISTS`, `ALTER TABLE ... ADD COLUMN IF NOT EXISTS`
- Constraint: dentro de `DO $$ BEGIN ... EXCEPTION WHEN duplicate_object THEN NULL; END $$;`
- `UPDATE` de correção de dados: sempre com uma condição que deixa de valer depois de aplicado (ex: `WHERE a.titulo IS NULL`), senão ele sobrescreve edições legítimas do usuário a cada reinício
- **Nunca** `DROP TABLE`, `DROP COLUMN` ou `DELETE` sem filtro

3. Toda rota async precisa de `asyncHandler`

```
router.get('/x', asyncHandler(async (req, res) => { ... }));
```

Express 4 não captura rejeição de Promise. Sem o wrapper, um erro de banco não tratado deixa a requisição pendurada para sempre — o usuário vê um "carregando" infinito, sem erro nenhum. Foi corrigido em massa no commit `60bf94c`.

4. Níveis 1 a 3 têm login; nível 4 não

`criaLogin = novoNivel <= 3 ( routes/public.js )`. Nível 4 é só contato: existe apenas em `apoiadores`, sem linha em `usuarios`.

5. O driver do `pg` devolve DATE como string

`db.js` registra `types.setTypeParser(1082, val => val)`. Sem isso, uma data de nascimento vira `Date` do JS, e ao serializar em JSON vira `"1969-02-07T00:00:00.000Z"` — quebrando a exibição. Não remova.

Autenticação e autorização

Fluxo de sessão

1. `POST /api/auth/login` com `{ login, senha, perfil }` — aceita login **ou** e-mail no campo `login`; `perfil` vem da aba escolhida na tela
2. Valida bcrypt, checa `data_desativacao` do candidato dono da rede
3. Assina JWT (12 h) e devolve em cookie `httpOnly` + `sameSite: lax` (`secure` só em produção)
4. O frontend usa `credentials: 'include'` em todo `fetch`
5. `authRequired` lê o cookie; aceita `Authorization: Bearer` como alternativa para chamadas via curl/script

Um `401` em qualquer chamada faz o frontend deslogar automaticamente (`api()` em `frontend/index.html`).

Camadas de permissão

Camada	Onde	O que faz
<code>authRequired</code>	<code>middleware/auth.js</code>	exige sessão válida
<code>requireRole(...)</code>	idem	restringe por perfil
<code>resolveWorkspace</code>	<code>middleware/workspace.js</code>	admin atua "como" candidato via <code>?as=<id></code>
<code>podeGerenciar()</code>	<code>routes/apoiadores.js</code>	valida que o alvo está na árvore de quem pede

`resolveWorkspace` define `req.effectiveId` / `req.effectivePerfil`. **Use sempre esses dois** em vez de `req.user.id` quando a ação for sobre a rede — é isso que faz a Central de Vagas funcionar.

As duas consultas de árvore

Estão no topo de `routes/apoiadores.js` e concentram toda a lógica de "quem vê quem":

- **SQL_ARVORE_CANDIDATO** — desce por `usuarios.criado_por` a partir do candidato e traz todos os `apoiadores` ligados a qualquer nó da árvore. Usada para candidato e admin.
- **SQL_ARVORE_LIDERANCA** — desce por `apoiadores.parent_id`. Necessária porque, depois que a liderança reorganiza a hierarquia, o `parent_id` passa a apontar para outro apoiador, e a primeira consulta perderia os níveis 3 e 4.

Regras de negócio

Limites da pirâmide

Cascata: `usuarios.limite_nivel1..4` do candidato → se `NULL`, cai nas variáveis `LIMITE_NIVEL1..4` → se ausentes, `50 / 30 / 15 / 10` (`utils/limites.js` + `config.js`).

Verificados em três pontos: cadastro autenticado, autcadastro por link (só no modo pessoal, onde existe um pai definido) e reorganização de hierarquia.

Duplicidade

`utils/duplicidade.js` bloqueia **dentro da mesma rede de candidato**: e-mail (em `usuarios`), telefone e título de eleitor (em `apoiadores` — essa tabela cobre todo mundo, com e sem login).

Reorganização de hierarquia

PUT `/api/apoiadores/:id` com `nivel` + `parent_id`. Validações, nesta ordem:

1. Só liderança ou candidato/admin podem reorganizar
2. Nível destino ∈ {2, 3, 4}
3. O responsável precisa estar exatamente **um nível acima**

4. Não se pode pendurar alguém sob um descendente dele mesmo (evita ciclo)
5. O novo responsável não pode estourar o limite do nível dele

Primeiro acesso obrigatório

`utils/termoStatus.js` responde duas perguntas a cada login:

- `precisaTrocarSenha` — `senha_temporaria = true`, quando a senha foi escolhida por outra pessoa
- `precisaAceitarTermo` — `termo-versao-aceita != TERMO-VERSAO` atual

Qualquer uma verdadeira bloqueia o painel até ser resolvida. **Admin fica de fora** — não opera dados de terceiros.

Encerramento de contrato

`data_desativacao` no candidato. No login (`routes/auth.js`), se a data já passou, bloqueia **o candidato e toda a rede criada por ele**. Nada é apagado.

Recuperação de senha

Token aleatório de 32 bytes; o banco guarda só o **SHA-256** dele, com validade de 1 hora. O link vai por e-mail com o token em claro. Redefinir também zera `senha_temporaria` — sem isso a pessoa cairia na tela de primeiro acesso logo depois de já ter escolhido a senha.

Endpoints

Todos sob `/api`. [A] = exige sessão.

`/auth`

Método	Rota	Quem	O quê
POST	<code>/login</code>	público	login ou e-mail + senha + perfil
GET	<code>/me</code> [A]	qualquer	dados da sessão
POST	<code>/logout</code>	qualquer	limpa o cookie
POST	<code>/esqueci-senha</code>	público	envia link (exige e-mail cadastrado)
POST	<code>/redefinir-senha</code>	público	consome o token

/usuarios [A]

Método	Rota	Quem
GET	/	candidato, admin
GET	/verificar-login?login=	qualquer logado
POST	/	candidato, admin
PUT	/:id	candidato, admin
PUT	/:id/senha	candidato, admin
DELETE	/:id	candidato, admin

/apoiadores [A]

Método	Rota	Quem
GET	/	qualquer (a árvore muda conforme o perfil)
GET	/duplicados	candidato, admin
POST	/	liderança, apoiador
PUT	/:id	quem passa em <code>podeGerenciar</code>
PUT	/:id/senha	idem (alvo precisa ter login)
DELETE	/:id	idem

/admin [A] — só admin

GET /candidatos, POST /candidatos, PUT /candidatos/:id/senha, PUT /candidatos/:id/plano, PUT /candidatos/:id/login, PUT /candidatos/:id/email

/public — sem autenticação

Método	Rota	O quê
GET	/lideranca/:id	contexto do link pessoal
GET	/convite?candidato=&nivel=	contexto do link por nível
POST	/autocadastro	cria o cadastro (transação)

Estas três são a única superfície pública que escreve no banco. Qualquer alteração aqui merece atenção redobrada: validação de título, LGPD, duplicidade e limites são aplicados neste ponto.

/conta [A]

POST /aceitar-termo, PUT /senha (pede a atual), PUT /login (idem)

/config [A]

GET / (qualquer logado lê), PUT / (só candidato)

Saúde

GET /api/health → {"ok":true}

Frontend

Arquivo único, `frontend/index.html`. Organizado em blocos marcados por comentários `// ===`. Funções globais chamadas por `onclick` inline — **não é um padrão a ser "modernizado"**: é o que mantém o arquivo sem build.

Estado global: `currentUser`, `currentRole`, `APOIADORES`, `CONFIG`, `workspaceAdmin`.

Helpers do modo workspace (admin atuando como candidato) — use sempre estes ao adicionar tela nova:

```
papelEfetivo() // 'candidato' quando o admin abre um workspace
idEfetivo()    // id do candidato do workspace, ou do usuário logado
wsQuery()      // '?as=<id>' para anexar à URL da API
```

Armadilha de celular: nunca reescreva `input.value` a cada tecla

No Android (Gboard) e no iOS, o teclado digita em **modo composição**: mantém a palavra em aberto enquanto sugere correções. Se um handler `oninput` reescrever `this.value` nesse meio-tempo, **o teclado descarta a composição inteira e o campo fica vazio** — a pessoa digita e nada aparece. No desktop o bug não existe, porque teclado físico não usa composição.

Foi o que aconteceu com os campos de login (corrigido em `41b2acb`, agosto/2026). O padrão correto está em `mascaraLogin()`:

1. Sai da função se `ev.isComposing` (ou se `data-compondo === '1'`)
2. Ouve `compositionstart` / `compositionend` no `document` para cobrir campos criados dinamicamente
3. Só atribui `input.value` se o texto realmente mudou
4. Restaura a posição do cursor com `setSelectionRange`

Os campos de login carregam `data-login`, que é como os listeners delegados os encontram. **Ao criar qualquer campo novo com máscara, siga esse mesmo padrão** — inclusive as máscaras numéricas, que hoje escapam só porque `inputmode="numeric"` desliga as sugestões do teclado.

Infraestrutura

Por que Swarm e não Compose

O servidor já rodava outras stacks nesse modelo. O Swarm dá reinício automático, limites de memória, `docker service scale` e as labels que o Traefik lê.

Consequência: o Swarm **não builda imagens** — só baixa. Por isso `build.sh` roda no servidor. E como o Swarm compara imagens pela `tag` (`latest`), um "Update the stack" sozinho não troca o container; o `build.sh` resolve isso com `docker service update --force`.

Redes

- `network_public` (externa, criada pelo Traefik) — só o app está nela
- `redeapoio_internal` (`internal: true`) — Postgres e app. O banco não tem porta publicada e não alcança a internet

Variáveis de ambiente

Variável	Obrigatória	Efeito
<code>DOMAIN</code>	sim	Host das labels do Traefik
<code>DB_PASSWORD</code>	sim	senha do Postgres — gravada no volume no 1º boot
<code>JWT_SECRET</code>	sim	o app não sobe sem ela (<code>server.js</code> faz <code>exit(1)</code>)
<code>PGHOST/PGPORT/PGUSER/PGPASSWORD/PGDATABASE</code>	sim	lidas direto pelo driver
<code>PUBLIC_URL</code>	sim	usada nos links de e-mail e de autocadastro
<code>SMTP_*</code>	não	sem elas, recuperação de senha falha em silêncio
<code>TERMO_VERSAO</code>	não	padrão <code>1.0</code>
<code>LIMITE_NIVEL1..4</code>	não	padrão 50/30/15/10
<code>FRONTEND_DIR</code>	não	<code>/frontend</code> no container
<code>SUPABASE_*</code>	não	histórico — migração única já concluída

Variáveis discretas do Postgres em vez de uma `DATABASE_URL` montada: senhas fortes contendo `/ @ :` `#` quebrariam o parser de URL de conexão.

Limitações conhecidas e dívidas técnicas

Verificadas na leitura do código em 4 de agosto de 2026. Nada aqui está quebrado a ponto de impedir o uso — são pontos a resolver quando houver espaço.

1. IP registrado no aceite do termo é o do proxy, não o do visitante

Impacto: legal. `routes/public.js` e `routes/conta.js` gravam `req.ip` em `termos_aceite` como prova de consentimento (LGPD art. 8º, §2º). Mas o `server.js` nunca chama `app.set('trust proxy', ...)`, então o Express usa o IP do socket — que é o do container do Traefik. **Todos os aceites registrados até hoje têm IP interno do Docker**, sem valor probatório.

Correção (uma linha, em `server.js`, antes das rotas):

```
app.set('trust proxy', true); // Traefik é quem preenche o X-Forwarded-For
```

Depende de o Traefik publicar as portas em `mode: host` — que é como `infra/traefik-stack.yml` já faz, justamente por isso. Registros antigos não são recuperáveis.

2. "Nome do Candidato" e "Idealizadora" não são salvos

Em Configurações, os dois campos só alteram o objeto `CONFIG` na memória do navegador. `salvarConfig()` envia ao servidor apenas os limites. Recarregou, voltou ao valor fixo. Precisaria de colunas em `usuarios` e de inclusão no `PUT /api/config`.

3. Sem testes automatizados

Não há suíte. Toda validação é manual. O roteiro de teste está em [Rotinas de Manutenção e Correção](#).

4. Sem versionamento de migração

`001_init.sql` cresce e é reaplicado inteiro a cada boot. Funciona e é seguro enquanto tudo for idempotente, mas o boot fica gradualmente mais lento e um erro de idempotência pode corromper dados silenciosamente a cada reinício.

5. Sem rate limiting

`POST /api/auth/login` e `POST /api/public/autocadastro` aceitam requisições ilimitadas. Um `express-rate-limit` no login e no autocadastro seria a próxima melhoria de segurança mais valiosa.

6. Senha mínima de 4 caracteres

Escolha deliberada, pelo público do sistema. Vale saber que é o menor denominador de segurança em vigor.

7. `migrate.sh` e `SUPABASE_*` são resíduo histórico

A migração do Supabase foi concluída em julho de 2025. O script e as variáveis não têm mais uso — podem ser removidos numa limpeza futura.

8. O repositório vive dentro do Google Drive (só no Windows da autora)

O Drive injeta arquivos `desktop.ini` dentro de `.git/refs/`, e o Git passa a avisar `bad object refs/desktop.ini` em `git log --all` e `git branch -a`. É cosmético — commits e push funcionam. Limpeza:

```
Get-ChildItem .git -Recurse -Filter desktop.ini -Force | Remove-Item -Force
```

Não afeta o servidor: lá o clone é limpo.

Histórico resumido

O sistema nasceu como página única no Vercel com Supabase, e a `service_role key` estava **hardcoded no HTML publicado** — qualquer visitante com "ver código-fonte" tinha acesso de administrador ao banco.

A branch `1.0` reescreveu tudo: Node/Express + Postgres próprio, containerizado (commit `469c153`, julho/2025). Marcos desde então:

Commit	Entrega
<code>469c153</code>	migração Vercel+Supabase → Docker+Postgres
<code>4d512fd</code>	adequação ao Swarm + Traefik
<code>6c956d9</code>	cookie httpOnly, CEP nacional, recuperação por e-mail
<code>bb3759c</code>	admin abre workspace de qualquer candidato
<code>5d1344e</code>	correção do bug de "0 indicados" (invariante do id-espelho)
<code>9132ed7</code>	validação do dígito verificador do título de eleitor
<code>11aee34</code>	4º nível, LGPD versionada, plano e desativação de candidato
<code>3ba553a</code>	limites da pirâmide passam a persistir por candidato
<code>4029812</code>	design mobile: navegação estilo app
<code>711d738</code>	autocadastro vira login (níveis 2-3), data digitável
<code>bce98d4</code>	candidato gera links de cadastro por nível
<code>7de5947</code> , <code>6cb2653</code>	liderança redefine senha e edita login/e-mail da própria rede
<code>41b2acb</code>	correção do campo de login vazio no celular (composição do teclado)

Histórico completo: `git log --oneline 1.0`.